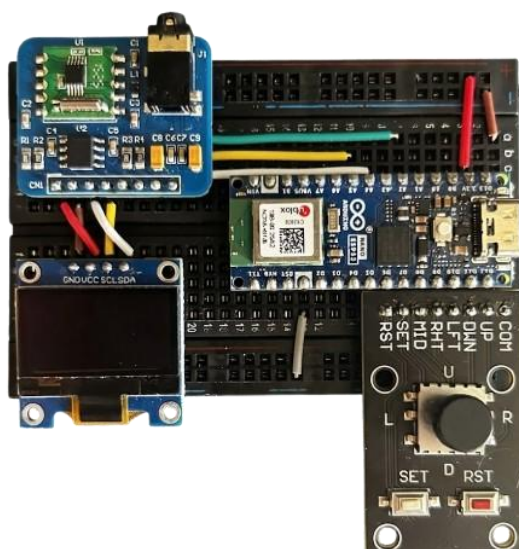


Departamento de Engenharia de Eletrónica e Telecomunicações e de Computadores

Licenciatura em Engenharia Eletrónica e Telecomunicações e de Computadores

Projeto Final de Curso

Relatório Final – P08



Recetor de Rádio FM com Controlo Digital

João Pedro Santos, N°51009

Presidente, Professor Doutor Fernando Manuel Ascenso Fortes

Arguente, Mestre Especialista Luís Miguel Rego Pires

Orientador, Professor Doutor Vítor Manuel de Oliveira Fialho

6º Semestre letivo 2025/2026

11 de Julho de 2026

Agradecimentos

Gostaria de expressar o meu profundo agradecimento ao meu orientador Professor Doutor Vítor Fialho, pela sua disponibilidade, paciência e partilha de conhecimento, que foram fundamentais para a concretização deste projeto.

Agradeço, igualmente ao arguente do júri, Professor Doutor Luís Pires, pelas valiosas dicas e sugestões oferecidas durante a apresentação e avaliação da proposta, as quais contribuíram significativamente para o enriquecimento e realização deste trabalho.

Um agradecimento especial ao Professor Mestre Pedro Sampaio, pelo apoio, disponibilidade e ajuda técnica demonstrados ao longo de todo o processo.

Por fim, agradeço à minha família e namorada pelo suporte incondicional, amor, compreensão e paciência demonstrados nos momentos de maior desafio.

Resumo

A radiodifusão FM continua a ser fundamental nas telecomunicações modernas, revelando-se crítica em situações de calamidade. Este projeto foca-se no desenvolvimento de um sistema de rádio FM com controlo digital, capaz de funcionar autonomamente como um medidor da qualidade do sinal recebido, conseguindo diferenciar estações válidas de ruído de fundo através do indicador RSSI (*Received Signal Strength Indicator*).

O sistema assenta no microcontrolador Arduino Nano ESP32 e no módulo Radio FM RDA5807M, que comunicam através do protocolo I²C. O utilizador interage com o sistema através de um *joystick* de cinco direções e também, a partir de um ecrã OLED, que apresenta em tempo real a frequência sintonizada, o nível de sinal (RSSI) e o volume. O firmware implementa uma máquina de estados que gere os modos de operação: rádio normal, menus de configuração e rotinas de varrimento de frequência com dois modos distintos (varrimento total da banda FM (80–108 MHz) e varrimento centrado em torno de uma frequência de interesse).

Além disso, o projeto apresenta uma interface de controlo remoto desenvolvida em Python, que comunica com o microcontrolador via UART (*Universal Asynchronous Receiver-Transmitter*). Esta disponibiliza uma *dashboard* gráfica com controlo total do rádio (volume, mute, frequência e scans) e visualização em tempo real do espetro RSSI vs. frequência, com exportação de dados para ficheiros PNG e TXT. O sistema comporta-se, assim, como um analisador de espetro FM de baixo custo.

Os resultados experimentais validaram a eficácia da aquisição de dados em tempo real, demonstrando que o sistema distingue corretamente estações ativas de frequências vazias, com uma diferença de RSSI mensurável mesmo em ambientes interiores com atenuação significativa.

Palavras-Chave: Rádio FM, RSSI, Arduino Nano ESP32, RDA5807M, I²C, OLED, UART, Python, Varrimento de Frequência, Dashboard

Abstract

FM radio broadcasting continues to be fundamental in modern telecommunications, proving critical in disaster situations. This project focuses on the development of a digitally controlled FM radio system capable of autonomously functioning as a meter of received signal quality, able to differentiate valid stations from background noise through the RSSI (Received Signal Strength Indicator).

The system is based on the Arduino Nano ESP32 microcontroller and the Radio FM RDA5807M module, which communicate via the I²C protocol. The user interacts with the system using a five-way joystick and an OLED screen, which displays the tuned frequency, signal level (RSSI), and volume in real time. The firmware implements a state machine that manages the operating modes: normal radio, configuration menus, and frequency scanning routines with two distinct modes (full FM band scan (80–108 MHz) and scan centered around a frequency of interest).

Furthermore, the project features a remote-control interface developed in Python, which communicates with the microcontroller via UART (*Universal Asynchronous Receiver-Transmitter*). This provides a graphical dashboard with full control of the radio (volume, mute, frequency, and scans) and real-time visualization of the RSSI spectrum vs. frequency, with data export to PNG and TXT files. The system therefore behaves as a low-cost FM spectrum analyzer.

The experimental results validated the effectiveness of real-time data acquisition, demonstrating that the system correctly distinguishes active stations from empty frequencies, with a measurable RSSI difference even in indoor environments with significant attenuation.

Key-Words: FM Radio, RSSI, Arduino Nano ESP32, RDA5807M, I²C, OLED, UART, Python, Frequency Scan, Dashboard

Índice

1.	Introdução e Enquadramento	1
2.	Estado da Arte e Trabalhos Relacionados	2
2.1.	Recetor RDS e a Integração em Redes IoT	2
2.2.	Implementação de Rádio FM em Dispositivos Utilitários	2
2.3.	Sistemas de Emergência e Monitorização de Espectro	2
2.4.	Sintonização Digital Baseada no Módulo TEA5767	2
3.	Descrição Teórica.....	3
3.1.	Modulação em Frequência (FM)	3
3.2.	Banda Estreita vs. Banda Larga FM	3
3.3.	Sistema de Dados (RDS)	3
3.4.	Propagação de Ondas VHF e Qualidade de Sinal.....	3
3.5.	RSSI – Received Signal Strength Indicator	4
3.5.1.	Média Logarítmica do RSSI.....	4
3.6.	Protocolo I ² C	4
4.	Arquitetura do Sistema.....	5
4.1.	Componentes de Hardware.....	5
4.1.1.	Microcontrolador – Arduino Nano ESP32	5
4.1.2.	Módulo de Rádio FM – RDA5807M	5
4.1.3.	Ecrã OLED – SSD1306.....	6
4.1.4.	Interface de Utilizador – 5-Way Navigation Switch	6
4.2.	Diagrama de Blocos e Interligações	7
4.3.	Firmware – Organização e Implementação	8
4.3.1.	Máquinas de estados	10
4.3.2.	Controlo do módulo Radio FM RDA5807.....	11
4.3.3.	Algoritmos de Scan	12
4.3.4.	Handler de Comandos UART	13
4.4.	Interface Python – Arquitetura	14
4.4.1.	Gráfico de Espectro (Matplotlib)	15
4.4.2.	Dashboard de Controlo (Tkinter)	16
4.4.3.	Exportação de Dados.....	17
5.	Testes Realizados	17
5.1.	Cenários e Metodologia de Teste.....	17
5.2.	Varrimento Total da Banda FM (80.0 – 108.0 MHz).....	17
5.3.	Varrimento Centrado.....	19
5.4.	Conclusão dos Testes.....	20
6.	Conclusões e Trabalho Futuro.....	20
6.1.	Conclusões.....	20
6.2.	Trabalho Futuro	20
7.	Referências.....	22
	Anexo.....	24

Índice de Figuras

Figura 1 - Arduino Nano ESP32	5
Figura 2 - módulo Radio FM RDA5907	5
Figura 3 - Esquema elétrico do breakout do módulo Radio FM RDA5807	6
Figura 4 - Display OLED SSD1306	6
Figura 5 - 5-Way Navigation Switch	6
Figura 6 - Diagrama de Blocos do Projeto.....	8
Figura 7 - Diagrama UML de Classes do sistema completo.....	9
Figura 8 - Diagrama de estados finitos do firmware.....	10
Figura 9 - Função Implementada para extração do nível de sinal (RSSI).....	11
Figura 10 - Implementação da função RadioVolUp().	11
Figura 11 - Implementação da função RadioVolDown().	11
Figura 12 - Implementação da função RadioMuteToggle()	11
Figura 13 - Fluxograma da função ScanTotal()	13
Figura 14 - Fluxograma da função ScanCentered()	13
Figura 15 - Diagrama de sequência: fluxo de um scan completo	15
Figura 16 - Dashboard de controlo remoto	16
Figura 17 - Espetro FM em ambiente Exterior	18
Figura 18 - Espetro FM em ambiente Interior	18
Figura 19 - Espetro FM em ambiente Estacionamento subterrâneo	19
Figura 20 - Varrimento Centrado Exterior	19
Figura 21 - Varrimento Centrado Interior	19
Figura 22 - Varrimento Centrado Estacionamento subterrâneo	19
Figura 23 – Ambiente Exterior escolhido para fazer os testes de scan	24
Figura 24 – Ambiente Interior escolhido para fazer os testes de scan	25
Figura 25 - Ambiente de Estacionamento subterrâneo escolhido para fazer os testes de scan	25

Índice de Tabelas

Tabela 1 - Comparação entre os microcontroladores.....	5
Tabela 2 - Mapeamento dos botões do joystick	7
Tabela 3 - Pinout completo do sistema	8
Tabela 4 - Estados da máquina de estados do firmware.....	10
Tabela 5 - Protocolo de comandos UART Python - Arduino	14

Índice de Acrônimos

AC – Alternating Current

AF – Alternative Frequency

AGC – Automatic Gain Control (Controle Automático de Ganho)

AM – Amplitude Modulada

BLE – Bluetooth Low Energy

CMOS – Complementary Metal-Oxide-Semiconductor

ESP32 – Microcontrolador da família Espressif Systems

FM – Modulação em Frequência

FSM – Finite State Machine (Máquina de Estados Finitos)

I²C – Inter-Integrated Circuit

IoT – Internet of Things

LCD – Liquid Crystal Display

LOS – Line of Sight

MPX – Multiplex

MQTT – Message Queuing Telemetry Transport

NBFM – Narrowband Frequency Modulation (Modulação de Frequência de Banda Estreita)

OLED – Organic Light-Emitting Diode

PCB – Printed Circuit Board

PNG – Portable Network Graphics

PS – Programme Service

RDS – Radio Data System

RSSI – Received Signal Strength Indicator

RT – Radio Text

RX – Receive

SCL – Serial Clock Line

SDA – Serial Data Line

SNR – Signal-to-Noise Ratio

SRAM – Static Random-Access Memory

TX – Transmit

TXT – Text

UART – Universal Asynchronous Receiver-Transmitter

UML – Unified Modeling Language

USB – Universal Serial Bus

VHF – Very High Frequency (Frequência Muito Alta)

WBFM – Wideband Frequency Modulation (Modulação de Frequência de Banda Larga)

1. Introdução e Enquadramento

Devido à sua ampla cobertura territorial, potência e reduzido consumo energético, a radiodifusão FM continua a ter um papel crítico na propagação de informação e entretenimento para a população. Além disto, têm-se evidenciado uma grande vulnerabilidade nas infraestruturas modernas de comunicação, notavelmente durante apagões elétricos e situações de calamidade, como ocorrido recentemente em Portugal [1]. Em cenários de emergência como estes, a análise da qualidade do sinal FM é estritamente necessária para avaliar a sua eficácia, que pode ser muito variável.

A qualidade da receção do sinal é dependente de diversos fatores, como a intensidade do sinal recebido, o ambiente envolvente (e.g. densidade da vegetação, habitações, etc.) e o tipo de antena utilizada. Atualmente, a utilização de circuitos integrados tem permitido o desenvolvimento de rádios compactos de baixo custo e facilmente manipulados por microcontroladores.

Neste contexto, o presente projeto tem como objetivo o desenvolvimento de um recetor de rádio FM com controlo digital, baseado no microcontrolador Arduino Nano ESP32 [2] e no módulo de receção FM, RDA5807M [3]. O sistema desenvolvido permite a sintonização de frequências na banda FM (80–108 MHz) e o controlo de volume, bem como a realização de varrimentos de frequência (*frequency scan*) para análise da intensidade do sinal recebido, quantificada pelo indicador RSSI (*Received Signal Strength Indicator*). A interação com o utilizador é assegurada por um *joystick* de cinco direções [4] e por um ecrã OLED [5], que apresenta em tempo real a frequência sintonizada, o nível de sinal (RSSI) e o volume atual.

Uma das componentes centrais deste projeto é a integração de uma interface de controlo remoto desenvolvida em Python, que comunica com o microcontrolador via porta série (UART). Esta interface disponibiliza uma *dashboard* gráfica que permite ao utilizador controlar o rádio remotamente: sintonizar frequências, ajustar o volume e lançar rotinas de varrimento, sem necessidade de recorrer aos botões físicos. Adicionalmente, os dados recolhidos durante o varrimento são visualizados em tempo real num gráfico de espetro (RSSI em função da frequência), podendo ser exportados para ficheiros PNG e TXT para análise posterior. Esta capacidade de varrimento e visualização espectral aproxima o sistema de um analisador de espetro FM de baixo custo, com potencial de aplicação em contextos de monitorização de cobertura de sinal, planeamento de frequências e operações de emergência.

2. Estado da Arte e Trabalhos Relacionados

Apesar da hegemonia dos serviços de *streaming* e das comunicações digitais, a radiodifusão em Frequência Modulada (FM) preserva uma importância estratégica inegável, sustentada pela sua enorme resiliência e baixo custo de infraestrutura. No panorama atual da engenharia, o foco mudou da eletrônica puramente analógica para sistemas onde o controlo e o processamento são maioritariamente digitais. Esta evolução permite que recetores compactos não só sintonizem áudio, como também extraiam dados via RDS (*Radio Data System*) e analisem a qualidade do sinal em tempo real. Este capítulo revê as soluções tecnológicas exploradas em projetos recentes, estabelecendo os pontos de contacto e as evoluções introduzidas no presente trabalho.

2.1. Recetor RDS e a Integração em Redes IoT

O potencial de integração da radiodifusão em redes de Internet das Coisas (IoT) foi o foco do trabalho desenvolvido por Peksinski *et al.* (2018) [6]. Os autores conceberam um recetor de rádio programável capaz de decodificar dados do sistema RDS (*Radio Data System*), utilizando como núcleo de processamento um microcontrolador ATMEGA88 [7] e o sintonizador RDA5807M [3]. A comunicação entre os componentes é realizada via I²C, permitindo ao microcontrolador aceder aos endereços de memória do sintonizador onde residem as tramas RDS, para posterior exibição num ecrã OLED. O circuito foi concebido com uma perspetiva de escalabilidade, incluindo pinos RX/TX dedicados para possibilitar a futura ligação de módulos externos de comunicação Wi-Fi e Bluetooth.

2.2. Implementação de Rádio FM em Dispositivos Utilitários

No âmbito da aplicação de sistemas embebidos em dispositivos de consumo, Emelu e Loongo (2025) documentaram a criação do "PulseEdge Clock-Radio" [8]. Este projeto consistiu no desenho de um rádio-relógio multifuncional gerido por um microcontrolador Raspberry Pi Pico [9] e programado em MicroPython. O sistema integra o módulo RDA5807M [3] para a receção FM e utiliza um amplificador PAM8403 [10] para fornecer o áudio a colunas de 4 Ω . A interação com o utilizador é garantida por botões de pressão táteis e um ecrã OLED, através dos quais é possível controlar o volume, sintonizar estações e configurar parâmetros de relógio e alarme, geridos com o apoio de um módulo RTC DS3231 [11].

2.3. Sistemas de Emergência e Monitorização de Espectro

A utilização da banda FM para difusão de avisos de segurança pública foi o tema da dissertação de Coelho (2023), que propôs um sistema de alerta denominado "Firetec Switch" [12]. Desenvolvido com o propósito de alertar condutores para a presença de fogos florestais, o protótipo recorre a um microcontrolador Arduino e a comutadores analógicos CMOS para atuar sobre a camada física da transmissão. Em emergências, o dispositivo corta o fluxo da emissão local de uma rádio e injeta um sinal de aviso, composto por áudio multiplexado (MPX) e comandos RDS, nomeadamente o código PTY 31 (destinado a alertas) e listas de frequências alternativas (AF). O estudo abordou ainda a importância do rácio sinal-ruído (SNR) e da integridade da portadora para garantir a receção correta dos alertas.

2.4. Sintonização Digital Baseada no Módulo TEA5767

Mishra *et al.* [13] apresentaram o desenvolvimento de um protótipo de recetor FM digital com o intuito de demonstrar as vantagens do processamento digital na sintonização de radiofrequência. A arquitetura proposta recorre ao circuito integrado Philips TEA5767 [14], o qual é interligado a um microcontrolador Arduino Uno [15] através do protocolo de comunicação I²C. Através deste barramento, o microcontrolador envia comandos de escrita para estabelecer a frequência pretendida e acionar a funcionalidade de procura automática, e utiliza comandos de leitura para extrair a informação relativa

ao nível do sinal recebido. A interface visual do sistema é assegurada por um ecrã LCD, que exhibe os dados recolhidos pelo módulo.

3. Descrição Teórica

3.1. Modulação em Frequência (FM)

A Modulação em Frequência (FM) é uma técnica de modulação analógica em que a informação a transmitir, o sinal modulante $m(t)$, é codificada como variações da frequência instantânea de uma portadora sinusoidal [16]. Ao contrário da Modulação em Amplitude (AM), onde a amplitude da portadora varia, na FM a amplitude permanece constante, conferindo ao sinal uma elevada imunidade ao ruído e às interferências de amplitude.

De acordo com Haykin [16], a expressão geral do sinal FM é dada por:

$$s(t) = A_c \cdot \cos \left[2\pi f_c t + 2\pi k_f \cdot \int_0^t m(t) dt \right]$$

onde A_c é a amplitude constante da portadora, f_c é a frequência central da portadora, k_f é a sensibilidade do modulador em frequência (Hz/V) e a integral representa a acumulação de fase proporcional ao sinal de informação. O desvio de frequência instantâneo face à portadora [16] é: $\Delta f = k_f A_m$.

Segundo Haykin [16] a razão entre o desvio de frequência (Δf) e a frequência máxima do sinal modulante (f_m) é chamada de índice de modulação (β):

$$\beta = \frac{\Delta f}{f_m}$$

3.2. Banda Estreita vs. Banda Larga FM

A FM classifica-se em dois regimes consoante o índice de modulação β [17]. Na **FM de banda estreita** (NBFM, $\beta \ll 1$), o espectro do sinal é semelhante ao da AM, ocupando uma largura de banda de aproximadamente $2 \cdot f_m$. É utilizada em comunicações de voz (rádio profissional, aeronáutica, serviços de emergência). Na **FM de banda larga** (WBFM, $\beta \gg 1$), utilizada na radiodifusão comercial (87,5–108 MHz), o desvio máximo é de ± 75 kHz e a largura de banda, calculada pela regra de Carson, é:

$$BT = 2(\Delta f + f_m) = 2 \cdot (75 + 15) = 180 \text{ kHz}$$

Na prática, os canais FM comerciais são espaçados de 200 kHz, correspondendo a incrementos de 100 kHz para cada passo de sintonização (valor usado nas rotinas de varrimento deste projeto). A largura de banda superior da FM de banda larga, em comparação com a AM, permite transmitir áudio de alta-fidelidade (até 15 kHz de largura de banda de áudio) e sinal estéreo.

3.3. Sistema de Dados (RDS)

O RDS (Radio Data System) é um protocolo de comunicação de dados digitais transmitido na subportadora de 57 kHz do sinal FM, definido pela norma IEC 62106 [18]. Os dados são organizados em grupos de 104 bits (4 blocos de 26 bits com correção de erros). As informações mais relevantes incluem o nome da emissora (PS, 8 caracteres), texto livre (RT, 64 caracteres), tipo de programa (PTY), frequências alternativas (AF) e hora (CT). O código PTY 31 identifica alertas de emergência [12].

3.4. Propagação de Ondas VHF e Qualidade de Sinal

A banda FM opera na gama VHF (87,5–108 MHz), com comprimentos de onda de 2,8 a 3,4 m [19]. A propagação é essencialmente por linha de vista (LOS) [19]. Os principais fenómenos que afetam o RSSI

são a atenuação no espaço livre (a potência recebida diminui com o quadrado da distância, potência $\propto 1/d^2$), efeitos de multitrajeto (reflexões e difrações), e absorção em estruturas de construção, principal fator nos testes realizados em ambiente interior [20].

3.5. RSSI – Received Signal Strength Indicator

No RDA5807M [3], o RSSI é um valor inteiro de 7 bits (gama 0–63, registo 0BH [15:9]) com escala logarítmica, acessível via I²C [3]. O chip integra um AGC digital [3] que controla o ganho do recetor em função da potência do sinal recebido.

Um valor elevado de RSSI indica a presença de uma portadora ativa na frequência sintonizada, correspondendo a um sinal com boa potência no ponto de receção. Valores baixos e consistentes ao longo de uma gama de frequências estabelecem o patamar de ruído (*noise floor*), que representa o nível mínimo de energia eletromagnética detetável pelo recetor na ausência de qualquer emissão útil. A diferença entre o RSSI de um sinal válido e este patamar denomina-se margem de deteção, quanto maior for esta margem, mais fiável é a identificação da portadora.

O valor de RSSI não depende exclusivamente da potência do transmissor, sendo influenciado por um conjunto de fatores associados ao canal de propagação: a distância entre emissor e recetor, os obstáculos no percurso (paredes, edifícios, relevo), fenómenos de multitrajeto (reflexões e difrações) e as características da antena recetora. Consequentemente, o mesmo canal pode apresentar valores de RSSI significativamente diferentes consoante o ambiente de medição, tornando este indicador uma métrica sensível e adequada para a caracterização da qualidade de cobertura de um sinal de rádio.

Dado que o RSSI é uma medição instantânea sujeita a flutuações provocadas pelo desvanecimento rápido do sinal (*fast fading*), a leitura de uma única amostra pode não ser representativa da potência média real recebida. Para obter uma estimativa mais estável e fiável, é prática comum recolher múltiplas amostras sucessivas e calcular um valor médio, conforme descrito na secção seguinte.

3.5.1. Média Logarítmica do RSSI

Como o RSSI é expresso numa escala logarítmica (dBm ou equivalente), a média aritmética direta de múltiplas amostras produz um resultado fisicamente incorreto, uma vez que opera sobre valores já convertidos para logaritmo. A abordagem matematicamente correta, denominada média logarítmica, divide-se em dois passos:

1. Conversão de cada amostra para escala linear de potência:

$$P_i = 10^{\frac{RSSI_i}{10}}$$

2. Média aritmética na escala linear e reconversão para escala logarítmica:

$$RSSI_{médio} = 10 \cdot \log_{10} \left[\left(\frac{1}{N} \right) \cdot \sum P_i \right]$$

Este procedimento garante que o valor resultante representa corretamente a potência média recebida, minimizando o efeito de amostras espúrias e produzindo uma estimativa estável e fisicamente consistente.

3.6. Protocolo I²C

O protocolo I²C é um barramento série síncrono de dois fios (SDA e SCL) desenvolvido pela Philips [21]. No sistema, o Arduino Nano ESP32 [2] opera como mestre e os escravos são o RDA5807M [3] (endereço 0x10) e o display SSD1306 [5] (endereço 0x3C). As resistências de pull-up de 4,7 kΩ garantem tempos de subida adequados a 3,3V [21].

4. Arquitetura do Sistema

Este capítulo descreve a arquitetura global do sistema desenvolvido, detalhando cada componente de hardware, o esquema de interligação, a organização do firmware e a interface Python. O sistema divide-se em três camadas funcionais: a camada de hardware (componentes físicos), a camada de firmware (software no Arduino) e a camada de interface (dashboard Python).

4.1. Componentes de Hardware

4.1.1. Microcontrolador – Arduino Nano ESP32

O Arduino Nano ESP32 [2] [Figura 1] é o elemento central do sistema, baseado no SoC ESP32-S3 da Espressif Systems: processador Dual-core Xtensa LX7 a 240 MHz, 512 KB de SRAM, 16 MB de Flash, Wi-Fi 4 e Bluetooth 5.0 LE. A escolha resultou da substituição do ARD-NanoV4-3V [22], insuficiente em memória (2 KB SRAM, 32 KB Flash) para carregar as bibliotecas necessárias.

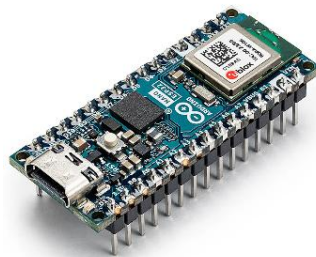


Figura 1 - Arduino Nano ESP32

O suporte para *Wi-Fi 4* e *Bluetooth LE*, abre uma vasta gama de possibilidades futuras para o tratamento de dados. Com acesso ao Wi-Fi, é possível usar sistemas de cloud para armazenar os dados e o Bluetooth permite, por exemplo, o controlo remoto do rádio.

Em suma, as principais diferenças encontram-se apresentadas na **Tabela 1**.

Tabela 1 - Comparação entre os microcontroladores

Microcontroladores	Clock	SRAM	Flash	Comunicações
ARD-NanoV4-3V	16MHz	2KB	32KB	Sem Wireless
Arduino Nano ESP32	240MHz	512KB	16MB	Wi-Fi e Bluetooth

4.1.2. Módulo de Rádio FM – RDA5807M

O módulo Radio FM RDA5807 [23] (baseado no circuito integrado **RDA5807M** [3]) foi desenvolvido pelo professor Pedro Sampaio sob a forma de uma PCB, como demonstrado na **Figura 2**.



Figura 2 - módulo Radio FM RDA5807

A **Figura 3** apresenta o esquema elétrico do módulo Radio FM RDA5807 [23], concebido sob a forma de uma PCB desenvolvida pelo professor Pedro Sampaio. Este módulo tem como principal componente o circuito integrado RDA5807 [3]. O chip é responsável pela recepção e desmodulação do sinal FM, sendo controlado pelo microcontrolador, Arduino Nano ESP32 [2], através do barramento de comunicação I²C. Como os sinais analógicos de saída do RDA5807 [3] (L-OUT e R-OUT) têm uma potência baixa, o módulo incorpora um amplificador de áudio estéreo dedicado, o TPA611A2DR [24]. Os sinais passam por condensadores de acoplamento AC antes de entrarem neste amplificador.

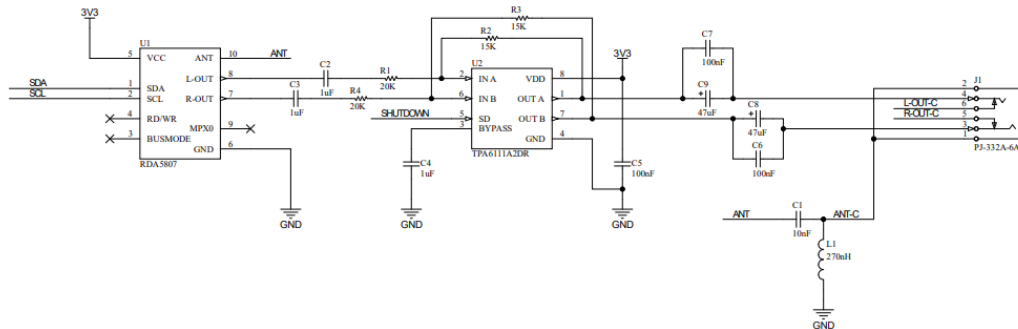


Figura 3 - Esquema elétrico do breakout do módulo Radio FM RDA5807

4.1.3. Ecrã OLED – SSD1306

O sistema de visualização baseia-se num ecrã OLED monocromático de 128×64 pixéis, gerido pelo controlador SSD1306 [5] [Figura 4], que comunica via interface I²C (endereço 0x3C) através da biblioteca Adafruit SSD1306 [25]. O *firmware* implementa quatro ecrãs distintos de interação: rádio normal, menu principal, seleção de frequência central e configuração de *scan*.



Figura 4 - Display OLED SSD1306

4.1.4. Interface de Utilizador – 5-Way Navigation Switch

O controlo do rádio foi implementado com a utilização do 5-Way Navigation Switch da JOY-it [4] [Figura 5]. Para otimizar a montagem, os pinos foram configurados no microcontrolador como *INPUT_PULLUP*. Desta forma, o sistema regista uma ação quando o nível lógico do pino transita para *low*.



Figura 5 - 5-Way Navigation Switch

O rádio tem dois modos, o Modo RADIO e o Modo Menus, nestes dois modos cada botão funciona de forma diferente. No Modo RADIO temos, os botões UP e DOWN controlam o volume do amplificador interno do rádio, incrementando ou decrementando o valor dentro dos limites do chip RDA5807M (0 a 15). Já os botões LEFT e RIGHT controlam a sintonização manual. Incrementa (RIGHT) ou decrementa (LEFT) a frequência em passos de 0.1MHz (100kHz). O botão SET funciona para trocar o modo de funcionamento do rádio. Já no Modo Menus, os botões UP e DOWN são usados para navegar nos menus e incrementar e decrementar valores quando necessário. Os botões LEFT e RIGHT funcionam para movimentar o cursor para a esquerda e para a direita respetivamente. Por fim os botões SET e RST, confirma/avança e retrocede/cancela, respetivamente. A **Tabela 2** resume o mapeamento dos botões.

Tabela 2 - Mapeamento dos botões do joystick

Botão	GPIO	Modo Radio	Modo Menus
UP	12	Volume +1	Navega para cima / incrementa
DOWN	11	Volume -1	Navega para baixo / decrementa
LEFT	10	Freq. -0.1 MHz	Move o cursor para a esquerda
RIGHT	9	Freq. +0.1 MHz	Move o cursor para a direita
SET	7	Acede ao Menu	Avança / Confirma
RST	6	-	Retrocede / Cancela

4.2. Diagrama de Blocos e Interligações

A arquitetura física do sistema está representada no diagrama de blocos seguinte [Figura 6]. O módulo Rádio FM RDA5807 [23] e o ecrã OLED SSD1306 [5] partilham o barramento I²C, recorrendo a um par de resistências de *pull-up* nas linhas SDA (*Serial Data*) e SCL (*Serial Clock*). Esta configuração assegura que o barramento partilhado opere com níveis lógicos estáveis de 3.3V, minimizando o ruído e prevenindo possíveis falhas de comunicação entre o Arduino e os periféricos.

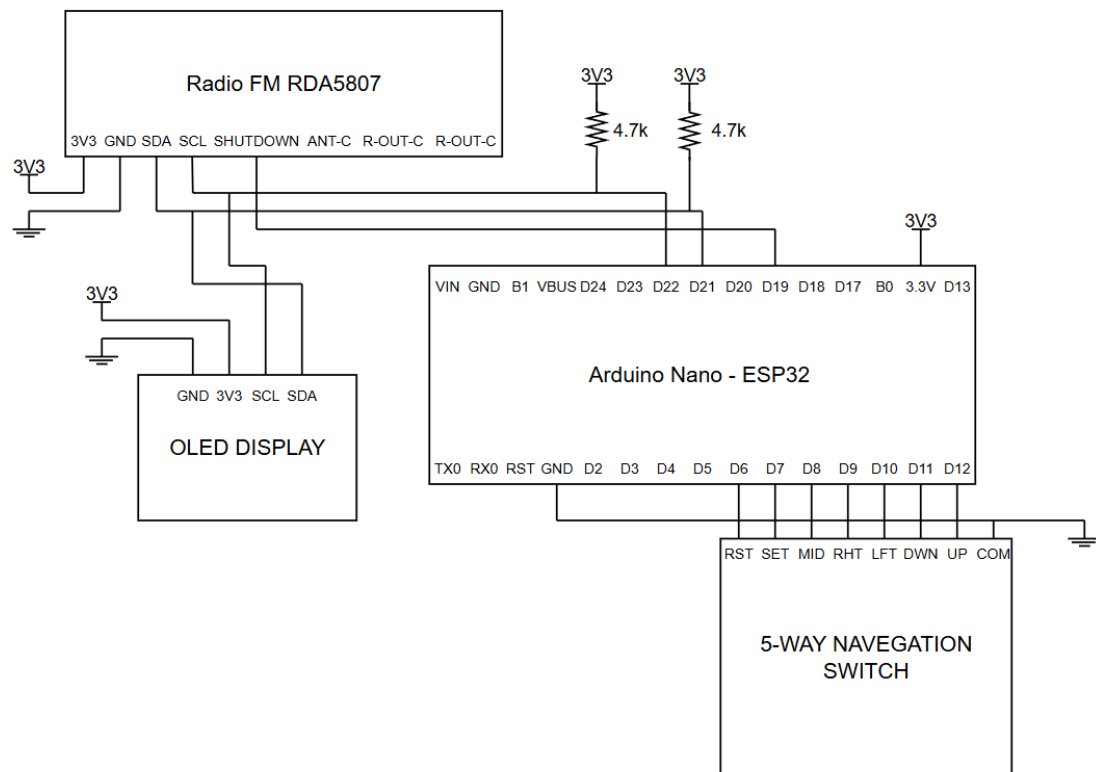


Figura 6 - Diagrama de Blocos do Projeto

Para além das ligações I²C acima referidas, o circuito integra um *joystick* de navegação de 5 vias (*5-way navigation switch*) [4] para interação com o utilizador, bem como uma interface de comunicação série via USB com o computador (para execução do *script* em Python). A **Tabela 3** sistematiza o *pinout* completo do sistema, detalhando a atribuição de todos os pinos do Arduino Nano ESP32 [2] e a respetiva função de cada periférico.

Tabela 3 - Pinout completo do sistema

Periférico	Pino Arduino	Função
Radio FM RDA5807	A4 (SDA)	Dados I ² C
	A5 (SCL)	Clock I ² C
OLED SSD1306	A4 (SDA)	Dados I ² C
	A5 (SCL)	Clock I ² C
Joystick UP/DWN/LEF/RHT/SET/RST	D12/D11/D10/D9/D7/D6	Input digital INPUT_PULLUP
PC (Python)	USB / TX0-RX0	UART 115200 baud

4.3. Firmware – Organização e Implementação

O *firmware* foi desenvolvido em C++ na plataforma Arduino IDE e o *software* de controlo em Python na plataforma Visual Studio Code. O código foi estruturado em módulos independentes para facilitar a manutenção. A **Figura 7** apresenta o diagrama de classes UML que ilustra a estrutura modular completa, evidenciando as relações entre os módulos Arduino, os módulos Python e as bibliotecas externas.

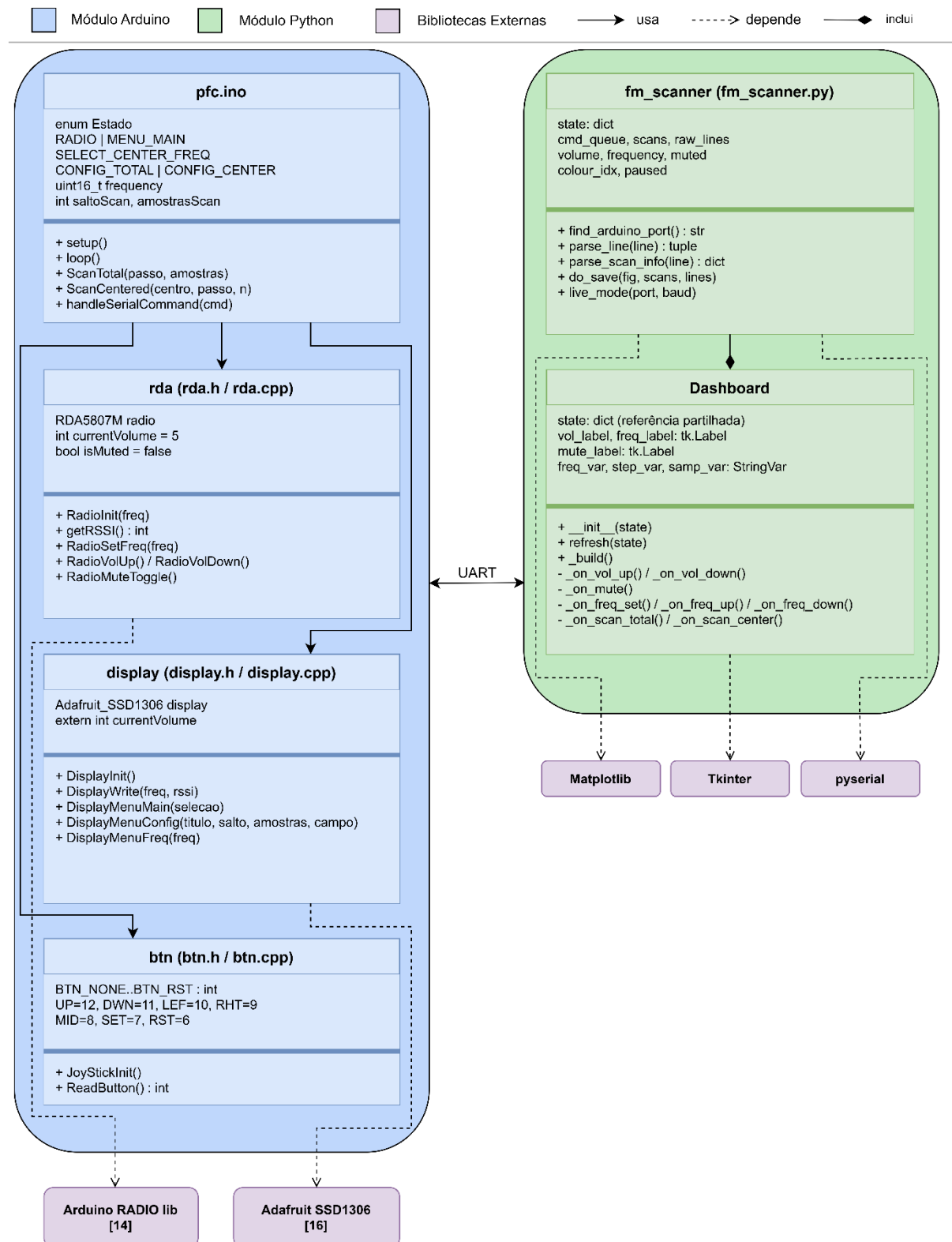


Figura 7 - Diagrama UML de Classes do sistema completo

4.3.1. Máquinas de estados

O ciclo de execução principal (*loop*) implementa uma Máquina de Estados Finita (FSM) composta por cinco estados principais. A **Tabela 4** detalha o propósito de cada estado e as respectivas transições, enquanto a **Figura 8** ilustra visualmente a lógica de comutação implementada.

Tabela 4 - Estados da máquina de estados do firmware

Estado	Descrição	Transições
RADIO	Modo Normal – OLED mostra a Freq. o RSSI e o Volume	MENU_MAIN (SET)
MENU_MAIN	Seleção do tipo de scan	CONFIG_TOTAL, SELECT_CENTER_FREQ, RADIO
SELECT_CENTER_FREQ	Seleção da frequência central	CONFIG_CENTER, MENU_MAIN
CONFIG_TOTAL	Config. Scan total (passo e amostras)	ScanTotal(), MENU_MAIN
CONFIG_CENTER	Config. scan centrado	ScanCentered(), SELECT_CENTER_FREQ

Estado do rádio Menu Principal Config. Total Config. Central Execução

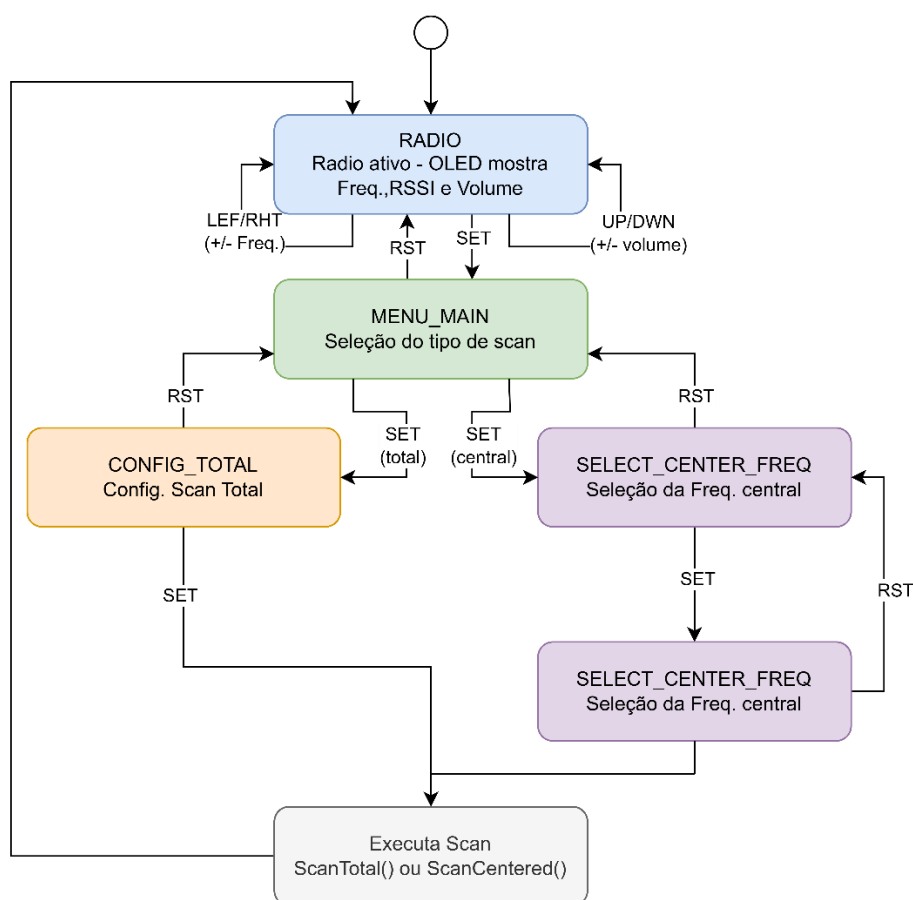


Figura 8 - Diagrama de estados finitos do firmware

4.3.2. Controlo do módulo Radio FM RDA5807

A interface de controlo e configuração do módulo Radio FM RDA5807 [23] é realizada exclusivamente através do barramento I²C. Para apaziguar a complexidade da manipulação direta dos registos internos do recetor, o firmware desenvolvido recorre a uma biblioteca de controlo (Arduino Radio Lib [26]) que gere o estado do dispositivo através da estrutura de dados *RADIO_INFO*.

Para a aquisição do nível de RSSI foi implementada uma função [Figura 9] que atualiza a estrutura de informação do rádio e devolve o valor do RSSI num dado instante.

```
int getRSSI(){
    RADIO_INFO info;
    radio.getRadioInfo(&info);
    return info.rssi;
}
```

Figura 9 - Função Implementada para extração do nível de sinal (RSSI)

Já para o controlo do volume foram criadas duas funções *RadioVolUp()* [Figura 10] e *RadioVolDown()* [Figura 11]. Estas garantem que os comandos de incremento e decremento respeitam os limites do chip (de 0 a 15), ativando o SHUTDOWN quando o volume for 0, prevenindo situações de ruído e de possível *overflow* ou *underflow* no envio de comandos I²C.

```
void RadioVolUp() {
    if (currentVolume < 15) {
        if (currentVolume == 0){
            digitalWrite(SHUTDOWN_PIN, LOW);
            delay(10);
            radio.setMute(false);
        }
        currentVolume++;
        radio.setVolume(currentVolume);

        Serial.print("Volume: ");
        Serial.println(currentVolume);
    }
}
```

Figura 10 - Implementação da função *RadioVolUp()*.

```
void RadioVolDown() {
    if (currentVolume > 0) {
        currentVolume--;

        if (currentVolume == 0){
            radio.setMute(true);
            digitalWrite(SHUTDOWN_PIN, HIGH);
        } else{
            radio.setVolume(currentVolume);
        }

        Serial.print("Volume: ");
        Serial.println(currentVolume);
    }
}
```

Figura 11 - Implementação da função *RadioVolDown()*.

Por último, para o controlo do mute na dashboard, foi criada a função *RadioMuteToggle()*, como demonstrado na Figura 12. Esta garante que o *toggle* do *mute* é realizado sem quaisquer problemas, tendo uma variável booleana 'isMuted' independente do volume.

```
void RadioMuteToggle() {
    isMuted = !isMuted;
    if (isMuted) {
        radio.setMute(true);
        digitalWrite(SHUTDOWN_PIN, HIGH);
        Serial.println("MUTE:ON");
    } else {
        radio.setMute(false);
        digitalWrite(SHUTDOWN_PIN, LOW);
        Serial.println("MUTE:OFF");
    }
}
```

Figura 12 - Implementação da função *RadioMuteToggle()*

Para garantir a precisão das medições durante os varrimentos, o firmware aplica a média logarítmica descrita na Secção 3.5.1, recolhendo N amostras de RSSI por canal e calculando o valor médio antes de o enviar via UART para a interface Python.

4.3.3. Algoritmos de Scan

O sistema implementa dois modos de varrimento distintos: o varrimento total (*ScanTotal()*) [Figura 13], que percorre toda a banda FM comercial (80–108 MHz) de acordo com um passo de frequência configurável; e o varrimento centrado (*ScanCentered()*) [Figura 14], que analisa o espectro numa margem em torno de uma frequência central previamente seleccionada (centro ± 10 passos, totalizando 21 pontos de análise).

Em ambos os modos, a rotina executada pelo microcontrolador para cada frequência alvo obedece às seguintes etapas sequenciais: sintonização da frequência, aplicação de um atraso (*delay*) de 100 ms para garantir a estabilização do circuito de *Automatic Gain Control* (AGC) interno do recetor, execução de N leituras do nível de RSSI (espaçadas por intervalos de 20 ms), cálculo da média logarítmica das amostras recolhidas e, por fim, a transmissão do valor processado via UART para o computador.

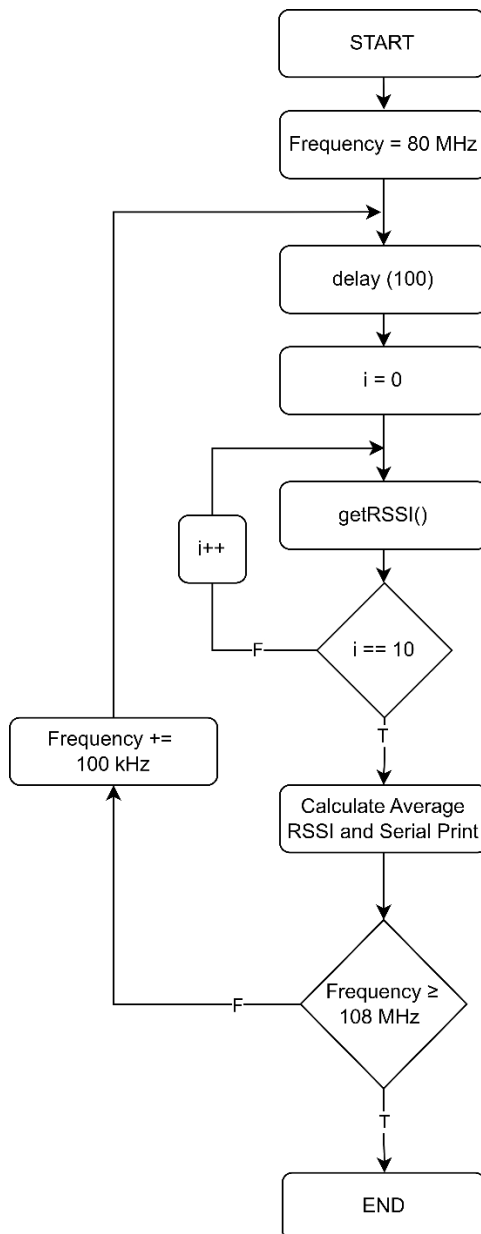


Figura 13 - Fluxograma da função ScanTotal()

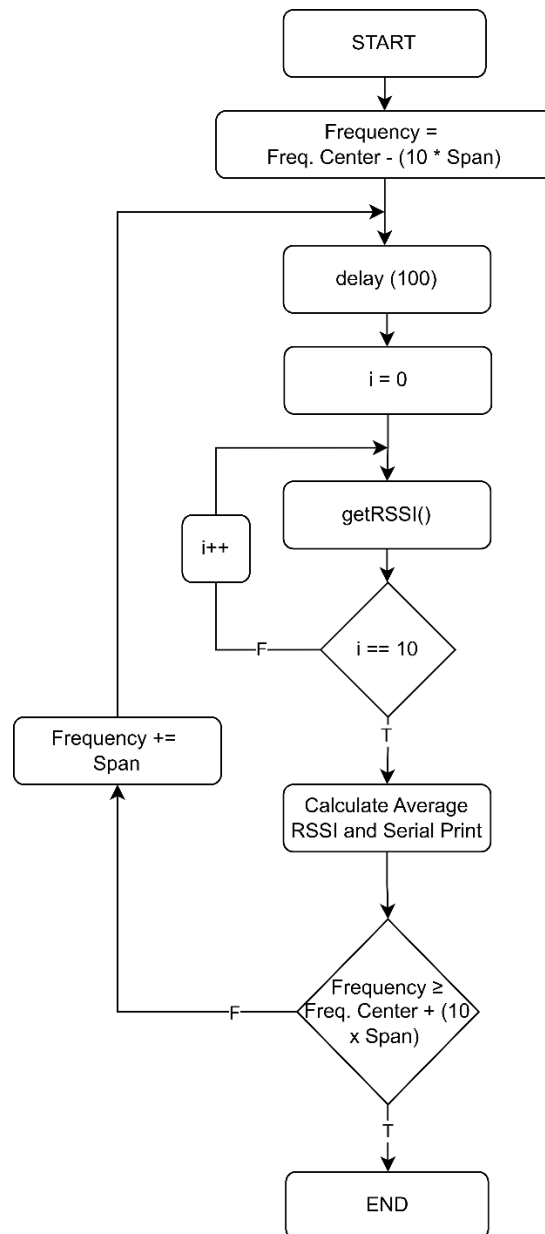


Figura 14 - Fluxograma da função ScanCentered()

4.3.4. Handler de Comandos UART

A comunicação bidirecional entre a *dashboard* em Python e o microcontrolador é gerida por um *handler* dedicado, implementado através da função *handleSerialCommand(cmd)* no módulo principal do Arduino (*pfc.ino*). Este bloco de código atua como o interpretador do sistema, monitorizando continuamente a porta série (UART) à procura de novas instruções enviadas pelo utilizador.

O protocolo de comunicação baseia-se na troca de mensagens em formato de *string* de texto. Quando uma nova instrução é recebida, o *handler* realiza o *parsing* da mensagem. Nos casos em que a instrução inclui parâmetros adicionais (como o comando *SCAN_CENTER*, que requer a frequência central, o passo e o número de amostras), os dados são extraídos utilizando o caractere ':' como delimitador. Após interpretar o comando, o microcontrolador invoca a função correspondente (e.g., *RadioVolUp()* para aumentar o volume ou *ScanTotal()* para iniciar um varrimento) e envia imediatamente uma resposta de

confirmação ou os dados recolhidos de volta para a interface. A **Tabela 5** detalha o conjunto de comandos suportados pelo protocolo.

Tabela 5 - Protocolo de comandos UART Python - Arduino

Comando	Parâmetros	Ação	Resposta
VOL +	-	<i>RadioVolUp()</i>	VOL: <n>
VOL -	-	<i>RadioVolDown()</i>	VOL: <n>
MUTE	-	<i>RadioMuteToggle()</i>	MUTE: ON/OFF
FREQ	<xxxx>	<i>RadioSetFreq(f)</i>	FRQ_OK: <xxxx>
SCAN_TOTAL	<pp>:<aa>	<i>ScanTotal()</i>	SCAN_INFO + dados
SCAN_CENTER	<ff>:<pp>:<aa>	<i>ScanCentered()</i>	SCAN_INFO + dados

4.4. Interface Python – Arquitetura

O *script* desenvolvido em Python constitui a camada de interface e controlo de alto nível do sistema, aproximando a experiência de utilização à de um instrumento de medida laboratorial. A **Figura 15** apresenta o diagrama de sequência que descreve o fluxo de execução completo, desde o momento em que o utilizador inicia um "SCAN TOTAL" na *dashboard* até à consequente atualização gráfica, ilustrando a comunicação bidirecional via UART.

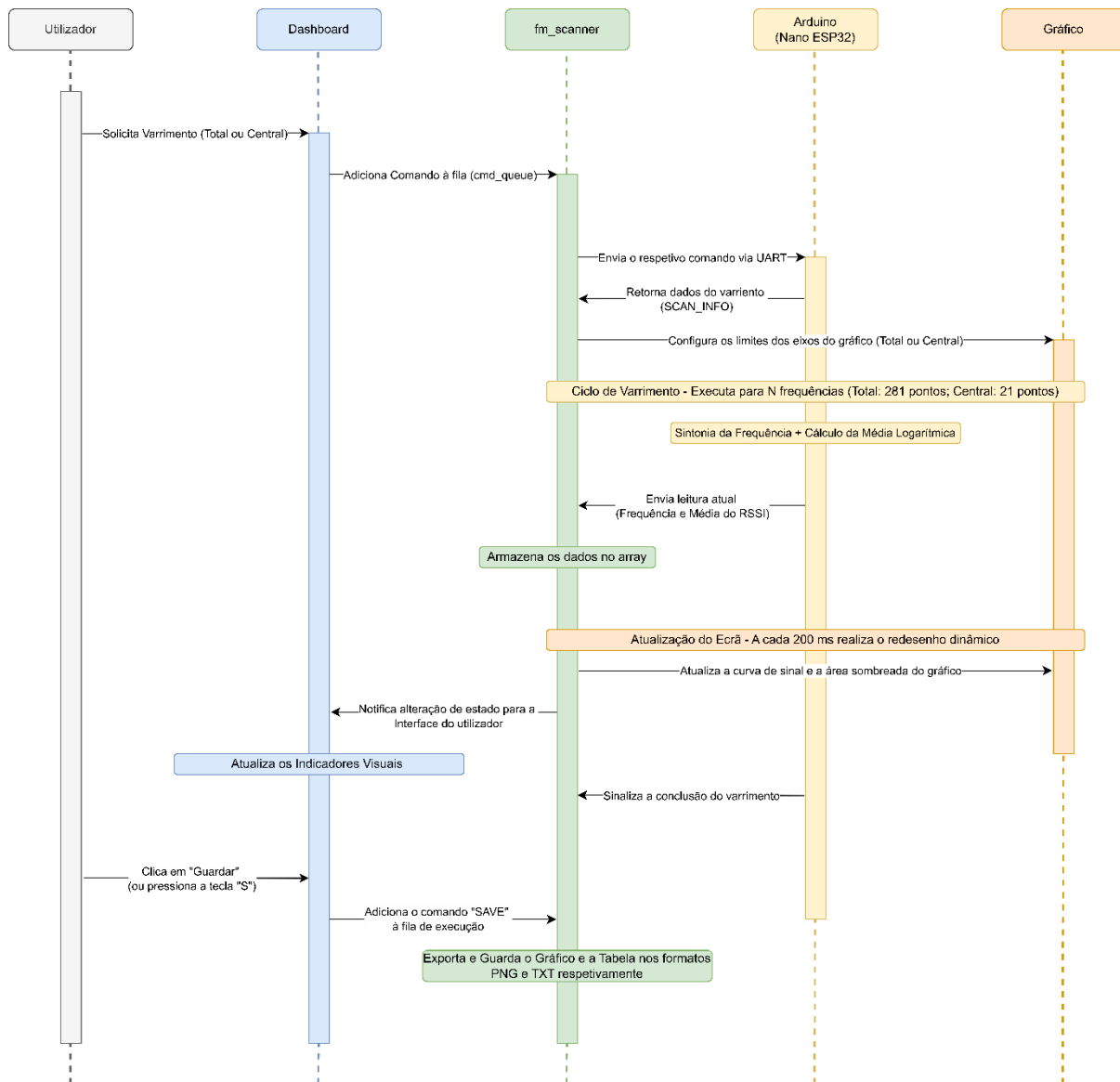


Figura 15 - Diagrama de sequência: fluxo de um scan completo

O elemento central desta arquitetura de *software* é a fila de comandos partilhada (*cmd_queue*), que separa a produção de comandos (pela interface Tkinter) do seu envio via ligação série (pela função de *loop* do *FuncAnimation*). Esta estratégia elimina potenciais conflitos de acesso aos recursos partilhados, dispensando a implementação de mecanismos complexos de sincronização, como *threads* ou *locks* explícitos.

4.4.1. Gráfico de Espectro (Matplotlib)

A componente de visualização gráfica baseia-se na biblioteca *Matplotlib*, sendo atualizada com uma periodicidade de 200 ms. O gráfico exibe a curva do RSSI médio para cada varrimento, acompanhada por uma área sombreada que ilustra os valores mínimo e máximo registados (indicando a variância temporal do sinal). O eixo das abcissas (frequências) é pré-configurado dinamicamente através dos dados recebidos (*SCAN_INFO*). O sistema suporta ainda a sobreposição de múltiplos varrimentos com diferenciação cromática, permitindo comparações visuais ao longo do tempo. É ainda possível colocar o gráfico em pausa (pressionando a barra de espaço ou o botão ‘Pause/Resume’ na dashboard) para observação detalhada, sem que ocorra perda dos dados, que continuam a ser adquiridos em segundo plano.

4.4.2. Dashboard de Controlo (Tkinter)

A interface gráfica de controlo (*dashboard*) [Figura 16] foi concebida recorrendo à biblioteca Tkinter. A janela encontra-se estruturada em quatro secções funcionais:

- **Volume:** Controlo de incremento, decremento e silenciamento (*mute*).
- **Frequência:** Configuração de uma frequência específica ou sintonização em passos de ± 0.1 MHz.
- **Scan:** Parametrização do passo e do número de amostras, bem como os botões de execução (Total ou Centrado).
- **Gráfico:** Ferramentas para alteração de cor da curva de análise, limpeza do ecrã (*reset*), pausa e exportação dos resultados.

Os indicadores visuais da interface (etiquetas de volume, frequência atual e estado do *mute*) são sincronizados iterativamente (através da função *dashboard.refresh()*), refletindo o estado real do microcontrolador.



Figura 16 - Dashboard de controlo remoto

4.4.3. Exportação de Dados

De forma a salvar a informação adquirida para posterior escrutínio, foi implementada uma funcionalidade de exportação de dados. Ao acionar o comando de gravação, o sistema compila a informação em memória e gera automaticamente dois ficheiros rotulados com um *timestamp* do momento da recolha (`fm_scan_YYYYMMDD_HHMMSS`):

- Um ficheiro de imagem (formato PNG), retendo o aspeto visual do gráfico.
- Um ficheiro de texto (formato TXT) contendo os dados tabulares estruturados com as frequências analisadas, o histórico de amostras de RSSI individuais recolhidas e a respetiva média logarítmica processada para cada varrimento.

5. Testes Realizados

Para validar o funcionamento do recetor de rádio FM e atestar a fiabilidade do sistema de aquisição de dados, procedeu-se a alguns testes práticos. O objetivo principal foi verificar a capacidade do sistema em sintonizar frequências ativas, quantificar a intensidade do sinal (*RSSI*) e comunicar estes dados em tempo real para a interface em Python, avaliando simultaneamente o impacto da envolvente física na propagação do sinal de rádio.

5.1. Cenários e Metodologia de Teste

Para observar os efeitos de atenuação multitrajeto e a absorção de sinal provocada por obstáculos (fenómenos descritos na Secção 3.4), o sistema foi posicionado em três ambientes com características de propagação distintas:

- **Exterior (*Outdoor*):** Zona ao ar livre.
- **Interior (*Indoor*):** No interior de um edifício.
- **Estacionamento (*Parking*):** Zona fechada/subterrânea.

As fotografias demonstrativas de cada ambiente encontram-se em **Anexo**.

Em cada local, executaram-se rotinas de varrimento total à banda comercial e rotinas de varrimento centrado, com a interface Python a gerar os respetivos ficheiros (TXT) e gráficos do espetro (PNG).

5.2. Varrimento Total da Banda FM (80.0 – 108.0 MHz)

O varrimento total, realizado com passos de 100 kHz e 10 amostras de RSSI por frequência, permitiu traçar o perfil espectral completo da região. A comparação dos três cenários revela discrepâncias significativas na intensidade do sinal recebido.

Ambiente Exterior: Como demonstrado na **Figura 17**, este cenário apresentou a melhor relação sinal-ruído (SNR). O gráfico gerado distingue claramente o ruído de fundo (flutuando em torno de um nível RSSI de 8 a 10) das portadoras das estações de rádio locais. Observam-se picos acentuados de sinal perfeitamente delineados, com o destaque para a frequência de 92.8 MHz, onde o recetor registou um nível máximo com a média logarítmica de 29 (numa escala máxima do recetor de 63).

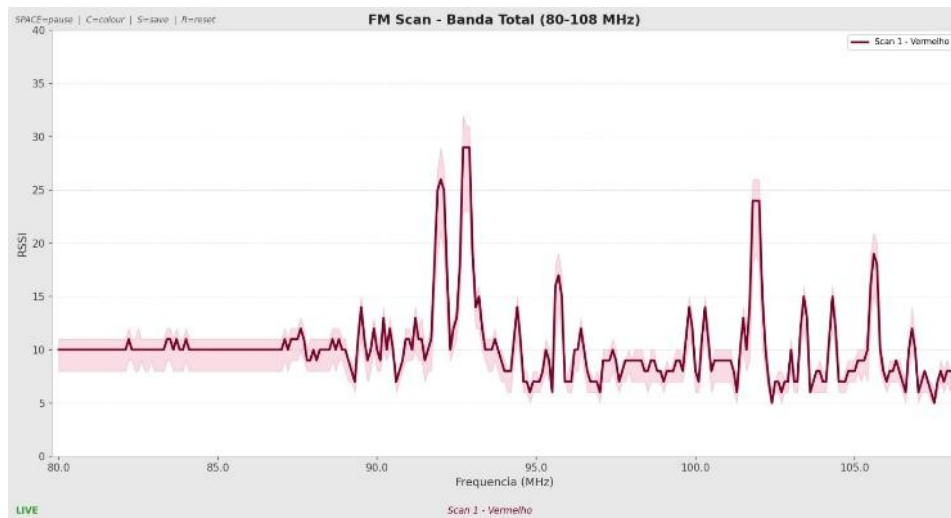


Figura 17 - Espectro FM em ambiente Exterior

Ambiente Interior: Como observado na **Figura 18** existe uma quebra notória na amplitude geral do sinal. O ruído de fundo mantém-se similar, mas a intensidade dos picos sofre uma forte atenuação. A estação localizada nos 92.8 MHz, que no exterior atingia um RSSI de 29, foi aqui captada com um valor médio de 16. Apesar da atenuação, o sistema de aquisição manteve-se sensível o suficiente para identificar claramente as estações mais potentes.

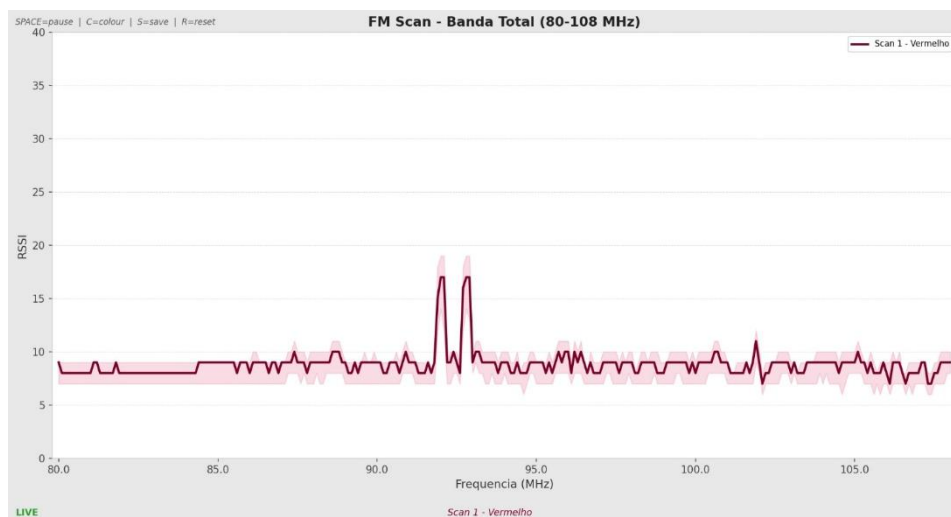


Figura 18 - Espectro FM em ambiente Interior

Ambiente de Estacionamento: Por fim, este teste demonstrou o bloqueio quase total da banda de rádio FM. O gráfico da **Figura 19** revela uma linha praticamente plana (oscilando entre níveis RSSI de 6 a 10), correspondendo exclusivamente a ruído eletromagnético de fundo. A frequência de 92.8 MHz obteve um RSSI residual de 9, tornando a desmodulação de áudio impossível.

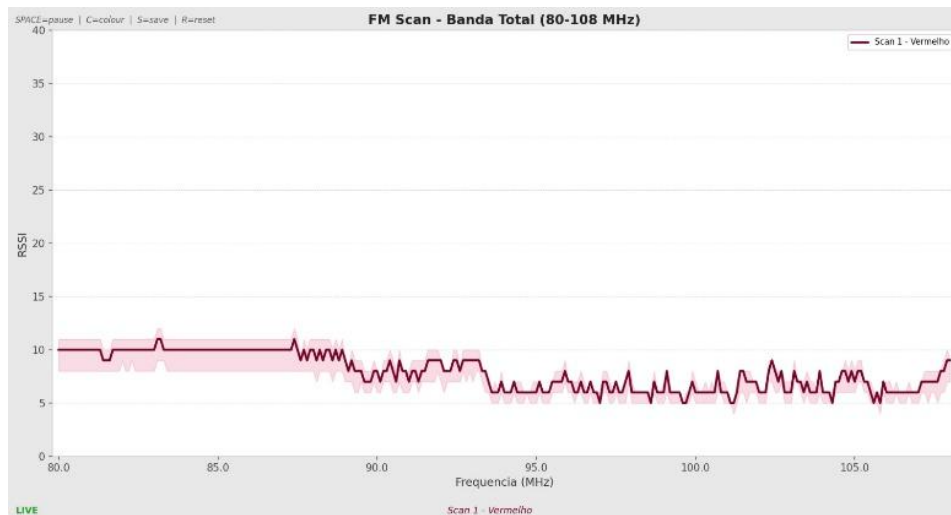


Figura 19 - Espectro FM em ambiente Estacionamento subterrâneo

5.3. Varrimento Centrado

Para atestar a granularidade do sistema, foi realizado um *scan* centrado focado na frequência de 93.2 MHz, nos mesmos três ambientes em que foram realizados os *scans* totais, analisando uma janela de ± 10 passos [Figura 20, Figura 21 e Figura 22] (92.2 MHz a 94.2 MHz).

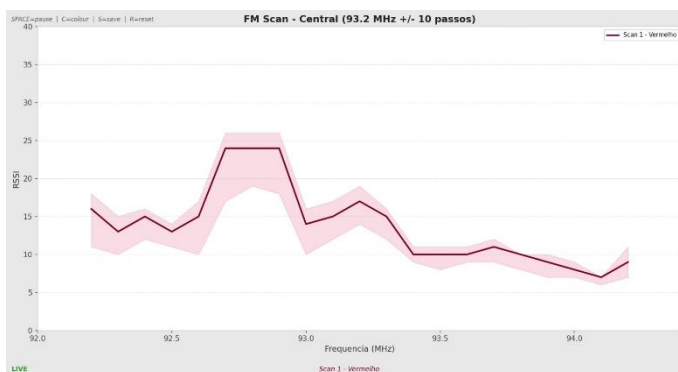


Figura 20 - Varrimento Centrado Exterior

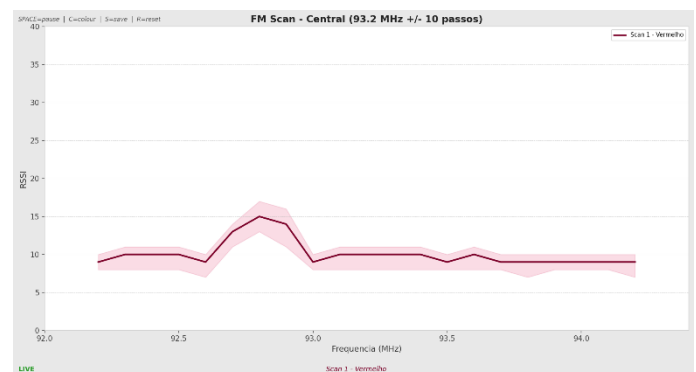


Figura 21 - Varrimento Centrado Interior

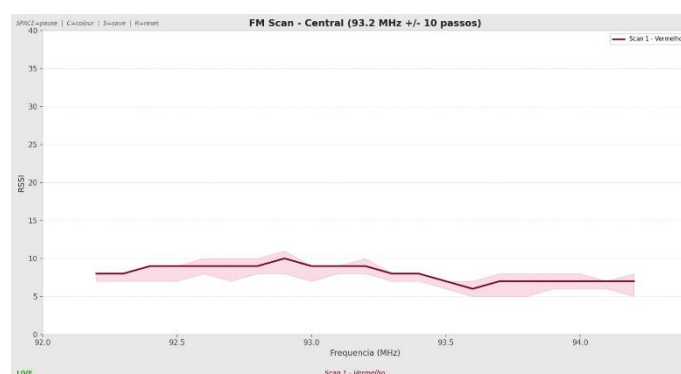


Figura 22 - Varrimento Centrado Estacionamento subterrâneo

Esta análise provou ser extremamente útil para observar a inclinação espectral (*slope*) de canais adjacentes. Analisando, por exemplo, os resultados do Varrimento Centrado Exterior, **Figura 20**, é possível observar a subida gradual do nível de sinal à medida que o rádio se aproxima do canal emissor mais forte na proximidade (92.8 MHz). O sistema registra um RSSI de 16 em 92.2 MHz, que escala

progressivamente até o valor de 24 nos 92.8 MHz, descendo novamente de forma contínua até estabilizar no ruído de fundo a partir dos 93.4 MHz. O sombreado gerado pela ferramenta *Matplotlib* (indicando os valores mínimo e máximo das amostras) ilustra perfeitamente a variância do sinal em tempo real sobre a portadora principal.

5.4. Conclusão dos Testes

Os testes experimentais comprovam que o hardware e o software desenvolvidos estão perfeitamente sincronizados. A fila de comandos UART assegurou que nenhum dado fosse perdido durante as extensas rotinas de aquisição (mais de 2800 amostras individuais por cada varrimento total). A interface revelou-se responsiva, e os ficheiros exportados demonstraram que o recetor RDA5807M [3], controlado digitalmente, é capaz de atuar como um equipamento de análise quantitativa viável para avaliação de qualidade de sinal em contexto de radiofrequência comercial.

6. Conclusões e Trabalho Futuro

6.1. Conclusões

O presente projeto teve como objetivo a conceção e o desenvolvimento de um recetor de rádio FM com controlo digital, dotado de capacidades de análise espectral da qualidade do sinal. Face aos resultados obtidos e à estabilidade demonstrada pelo protótipo final, conclui-se que os objetivos propostos foram integralmente cumpridos.

A nível de *hardware*, a integração do módulo Radio FM RDA5807 [23] com o microcontrolador Arduino Nano ESP32 [2] e com os periféricos de interface local (ecrã OLED [5] e *joystick* [4]) revelou-se eficaz. A utilização do barramento I²C garantiu uma comunicação robusta e sem perdas de informação. A nível de *firmware*, a implementação de uma máquina de estados finita permitiu gerir de forma fluida a transição entre o modo de rádio tradicional e os menus de configuração e varrimento.

A componente de *software* de alto nível, desenvolvida em Python, constituiu uma mais-valia substancial para o sistema. A criação de uma *dashboard* interativa e a comunicação bidirecional assíncrona via UART transformaram um simples recetor num analisador de espectro FM de baixo custo. Os testes práticos realizados em diferentes ambientes (exterior, interior e estacionamento subterrâneo) validaram a precisão do sistema de aquisição. O equipamento demonstrou capacidade para quantificar a intensidade do sinal (*RSSI*), distinguindo perfeitamente portadoras ativas de ruído de fundo, e evidenciou, com clareza gráfica, os fenómenos físicos de atenuação.

Em suma, o projeto resultou numa ferramenta funcional, didática e com potencial de expansão, que alia eletrónica de radiofrequência clássica a métodos modernos de processamento e visualização de dados.

6.2. Trabalho Futuro

Apesar de o protótipo cumprir plenamente os requisitos iniciais, a arquitetura modular adotada e as capacidades nativas do microcontrolador escolhido abrem portas a diversas melhorias futuras:

1. **Comunicação Sem Fios (IoT):** Uma vez que o Arduino Nano ESP32 [2] possui suporte nativo para Wi-Fi e Bluetooth (BLE), uma evolução natural do projeto seria a transição da comunicação série (cabo USB) para um protocolo *wireless* (e.g. MQTT). Isto permitiria colocar o rádio num local remoto (e.g., no telhado ou numa torre) e monitorizar o espectro FM a partir de um computador ou dispositivo móvel em qualquer parte do mundo.
2. **Descodificação de Dados RDS (*Radio Data System*):** O circuito integrado RDA5807M [3] suporta nativamente a extração de pacotes RDS. Uma atualização ao *firmware* permitiria descodificar estes pacotes para extrair dados úteis, tais como o nome da estação emissora (PS),

rádio texto (RT) ou sinalizações de alerta de emergência (PTY 31 [12]), exibindo-os tanto no ecrã OLED local como na interface Python.

3. **Integração de Áudio Externo:** De momento, o áudio é fornecido diretamente pelo módulo Radio FM RDA5807 [23] através de uma saída para auscultadores (*jack* de 3.5mm). O projeto poderia ser expandido com a adição de um circuito amplificador de áudio dedicado (e.g. PAM8403) e colunas integradas, melhorando a experiência de utilização no modo de rádio normal.
4. **Análise Automática de Dados com *Machine Learning*:** Tirando partido da rotina de exportação de dados estruturados em ficheiros TXT já implementada, seria possível recolher um *dataset* alargado de diferentes ambientes. Com esses dados, poder-se-ia treinar um modelo de classificação em Python capaz de identificar automaticamente o tipo de ambiente (aberto, fechado, com elevada interferência) apenas pela análise autónoma do ruído de fundo e da dispersão do RSSI.

7. Referências

[1] Joana Raposo Santos, Andreia Martins, Inês Moreira Santos, Carlos Santos Neves, Cristina Sambado, Apagão. Corte de energia elétrica atinge Portugal e outros países europeus, RTP, 29 abril 2025.

https://www.rtp.pt/noticias/mundo/apagao-corte-de-energia-eletrica-atinge-portugal-e-outros-paises-europeus_e1650845

[2] Arduino, “Arduino Nano ESP32 User Manual”, abril 2026.

<https://docs.arduino.cc/resources/datasheets/ABX00083-datasheet.pdf>

[3] RDA Microelectronics, “RDA5807M: Single-Chip Broadcast FM Radio Tuner”, maio 2011.

https://cdn-shop.adafruit.com/product-files/5651/5651_tuner84_RDA5807M_datasheet_v1.pdf

[4] JOY-it, “5-Way Navigation Switch Datasheet”, 2025.

https://www.joy-it.net/files/files/Produkte/COM-5WS/COM-5WS_Datasheet-EN_2025-08-25.pdf

[5] Solomon Systech Ltd., “SSD1306: 128x64 Dot Matrix OLED/PLED Segment/Common Driver with Controller”, abril 2008.

<https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>

[6] J. Peksinski et al., "Programmable FM Radio Receiver with RDS Decoder", Int. Conf. on Electronics, 2018.

[7] Atmel, “ATmega48/88/168 Datasheet”, julho 2014.

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7530-Automotive-Microcontrollers-ATmega48-ATmega88-ATmega168_Datasheet.pdf

[8] C. Emelu e M. Loongo, "PulseEdge Clock-Radio: A Multifunctional Embedded FM Radio System", Technical Report, 2025.

[9] Raspberry Pi, “Raspberry Pi Pico Datasheet” 2024.

<https://pip-assets.raspberrypi.com/categories/610-raspberry-pi-pico/documents/RP-008307-DS-1-pico-datasheet.pdf>

[10] Diodes Inc., “PAM8403: 3W Stereo Class-D Audio Amplifier”, setembro 2020.

<https://www.diodes.com/assets/Datasheets/PAM8403.pdf>

[11] Maxim Integrated, “DS3231: Extremely Accurate I²C-Integrated RTC with TCXO/Crystal” Rev.5 março 2015.

<https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>

[12] R. Coelho, "Firetec Switch: Sistema de Alerta de Emergência por Rádio FM", Dissertação, 2023.

[13] P. Mishra et al., "Digital FM Receiver Design Using Microcontroller and I2C Interfacing Techniques", ICSP 2016. DOI: 10.1049/cp.2016.1456

[14] NPX, “Low-power FM stereo radio for handheld applications” TEA5767 datasheet, janeiro 2007

<https://cdn.sparkfun.com/assets/4/5/f/a/d/TEA5767.pdf>

- [15] Arduino, “Arduino UNO User Manual”, julho 2026
<https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>
- [16] Haykin, S. (2001) *Communication Systems*. New York: John Wiley & Sons.
- [17] Couch, L.W. (2013) *Digital and Analog Communication Systems*. Pearson.
- [18] IEC 62106-1:2018, "Radio Data System (RDS) — Part 1: FM RDS Standard". Geneva: IEC, 2018.
- [19] ITU-R BS.412-9, "Planning standards for terrestrial FM sound broadcasting". Geneva: ITU, 1998.
- [20] J. G. Proakis e M. Salehi, *Communication Systems Engineering*, 2ª ed. Upper Saddle River: Prentice Hall, 2002.
- [21] NXP, "I2C-bus specification and user manual", Rev. 7.0, 2021.
<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [22] JOY-it, "ARD-NanoV4-3V Datasheet", 2025.
https://joy-it.net/files/files/Produkte/ARD-NanoV4-3V/ARD-NanoV4-3V_Datasheet-EN_2025-06-26.pdf
- [23] P. Sampaio, "Radio FM RDA5807 — PCB Breakout Module". Lisboa: ISEL, 2025.
<https://2526moodle.isel.pt/pluginfile.php/1304471/course/section/128904/RadioFM-Rev25.07.PDF>
- [24] Texas Instruments, “TPA6111A2 150-mW Stereo Audio Power Amplifier”, dezembro 2020
https://www.ti.com/lit/ds/symlink/tpa6111a2.pdf?ts=1783723534459&ref_url=https%253A%252F%252Fwww.google.com%252F
- [25] Adafruit Industries, "Adafruit SSD1306 OLED Library", GitHub, 2024.
https://github.com/adafruit/Adafruit_SSD1306
- [26] mathertel, "Arduino Radio Library", GitHub, 2024.
<https://github.com/mathertel/Radio>

Anexo



Figura 23 – Ambiente Exterior escolhido para fazer os testes de scan

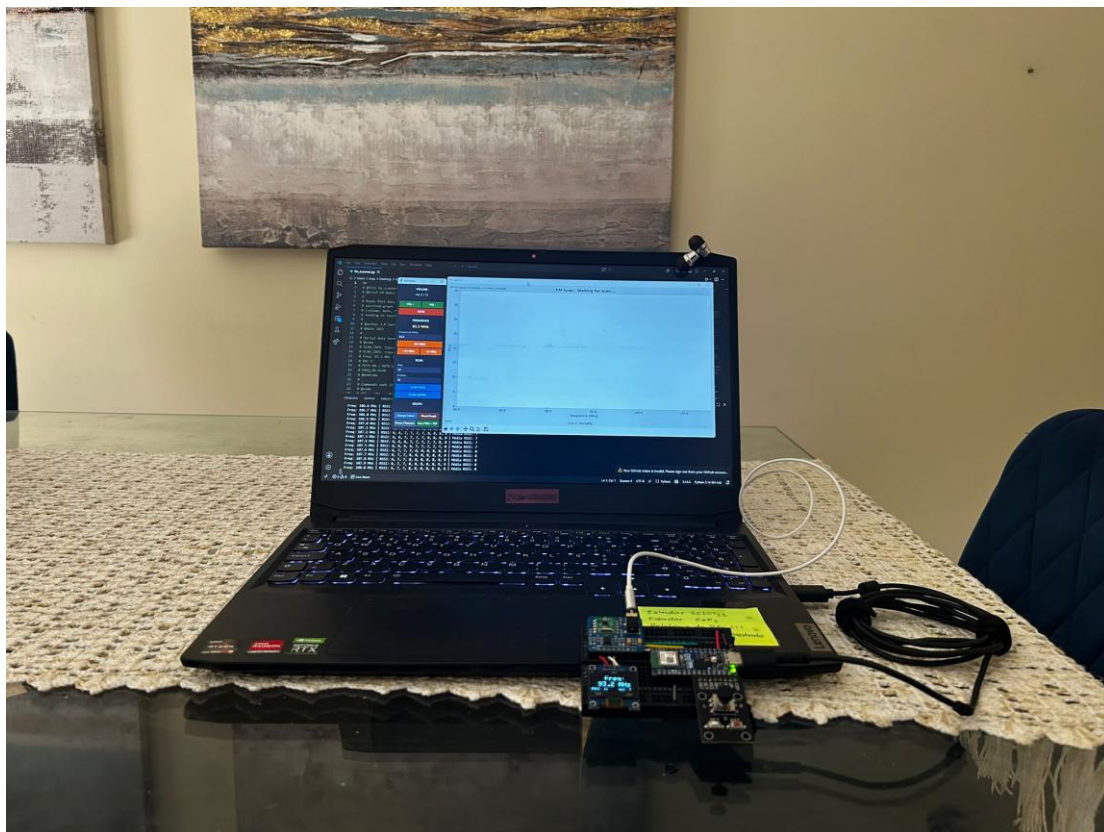


Figura 24 – Ambiente Interior escolhido para fazer os testes de scan

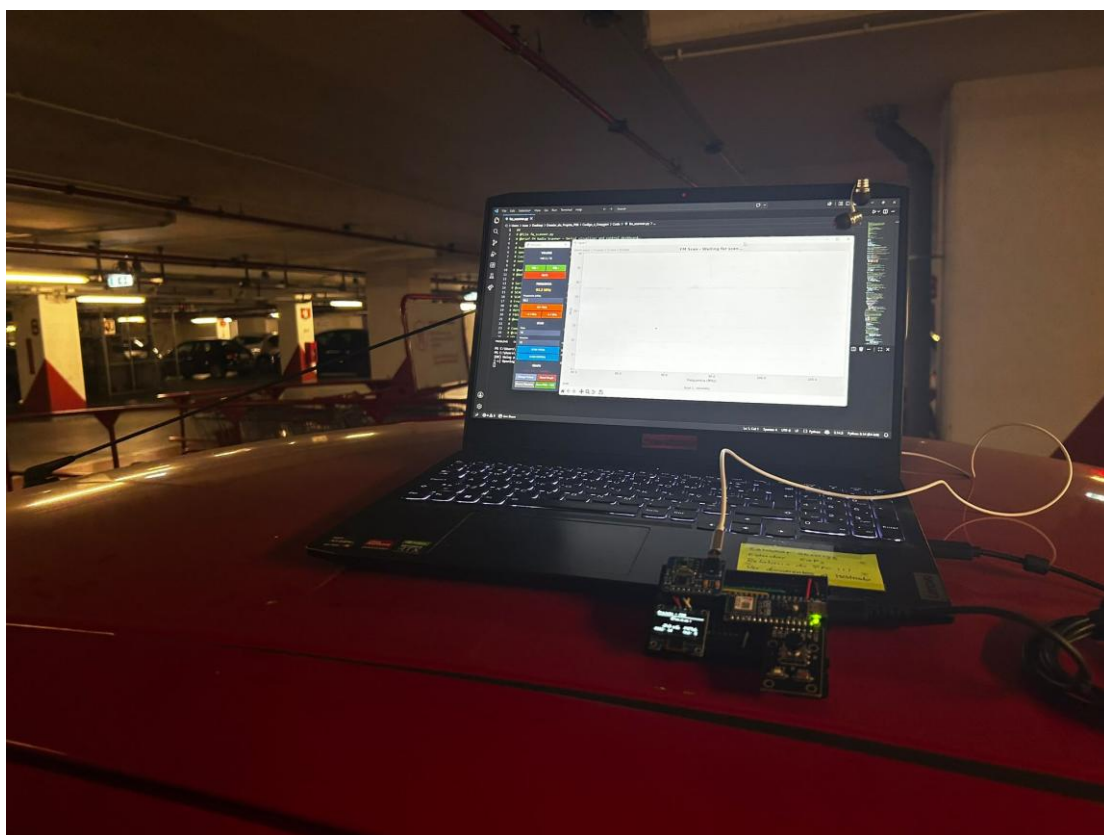


Figura 25 - Ambiente de Estacionamento subterrâneo escolhido para fazer os testes de scan